



Sparse R-CNN: End-to-End Object Detection with Learnable Proposals (CVPR2021)

(CABK505J)

이미지처리팀(Detection)

김병현(발표자), 김선옥, 김현진, 박재완, 안종식, 이찬혁, 현청천, 홍은기

Sparse R-CNN

❖ Sparse R-CNN

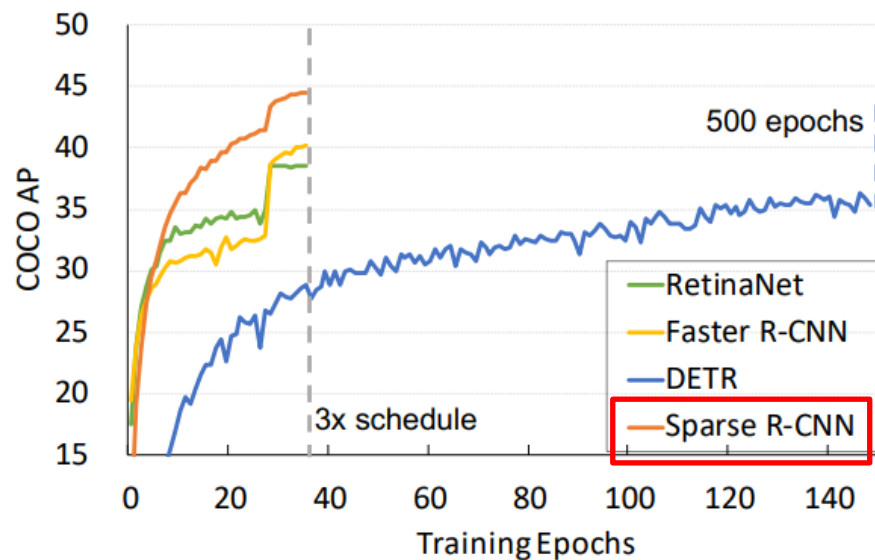
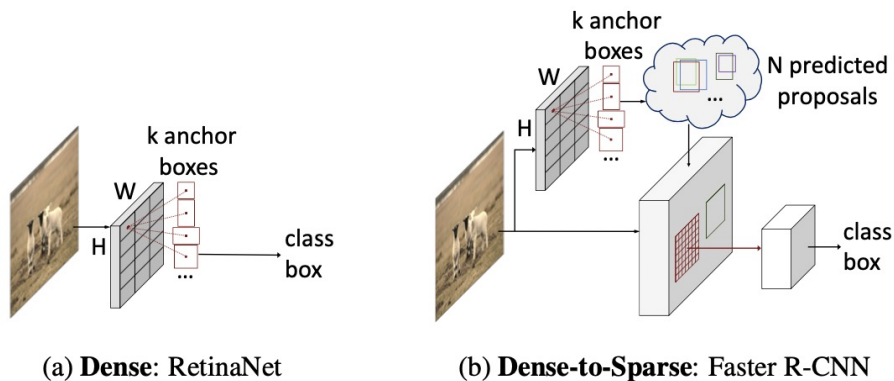
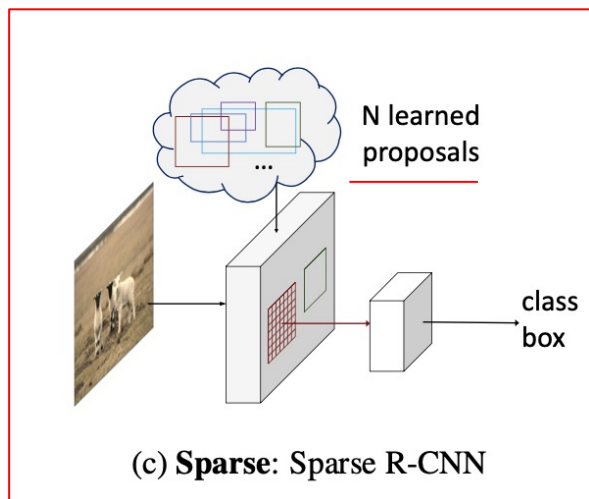


Figure 2 – Convergence curves of RetinaNet, Faster R-CNN, DETR and Sparse R-CNN on COCO val2017 [24]. Sparse R-CNN achieves competitive performance in terms of training efficiency and detection quality.

Introduction

❖ Current Paradigm of Region Proposal

⌘ **Dense** region proposal methods

1. Intuitive Algorithm
2. High Accuracy
3. Fast Inference Speed



YOLO series
SSD
Faster R-CNN
....

Introduction

❖ Current Paradigm of Region Proposal

❑ Dense Methods

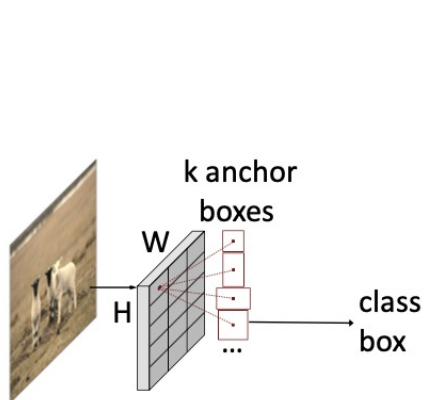
- YOLO, SSD, RetinaNet ...

❑ Dense-to-sparse methods

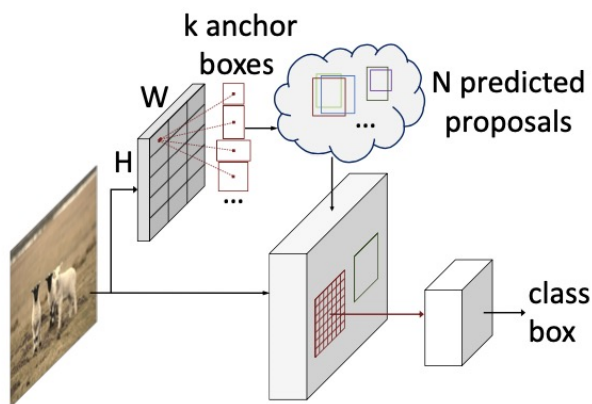
- Faster R-CNN...

❑ Sparse-to-Dense methods

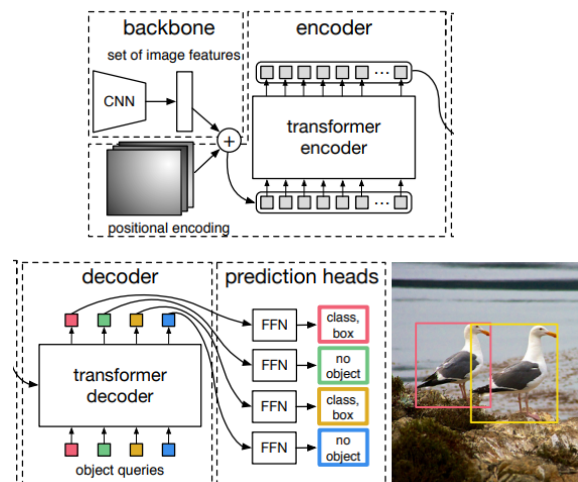
- DETR



(a) **Dense:** RetinaNet



(b) **Dense-to-Sparse:** Faster R-CNN



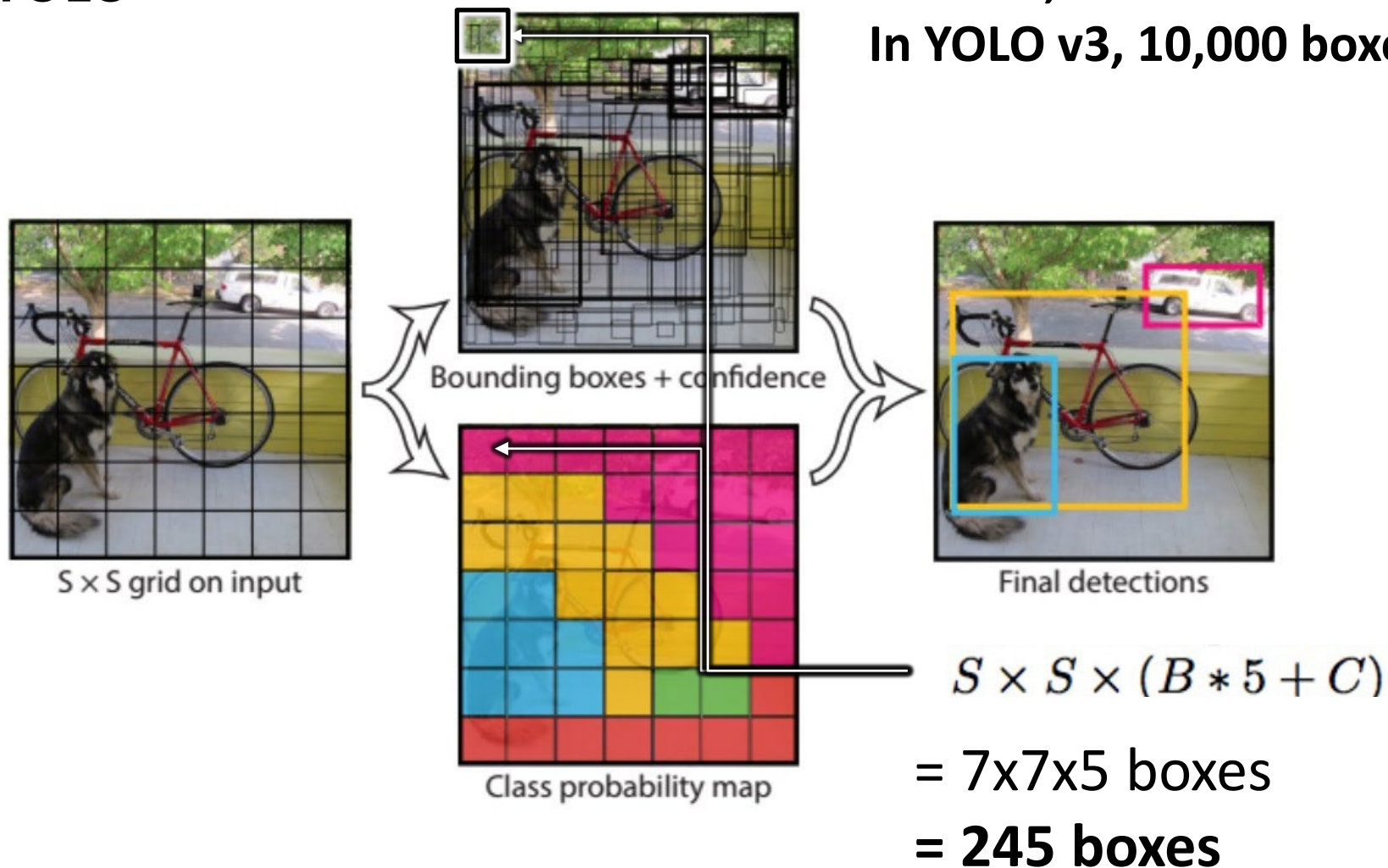
(c) **Sparse-to-Dense :** DETR

Dense Method

❖ YOLO

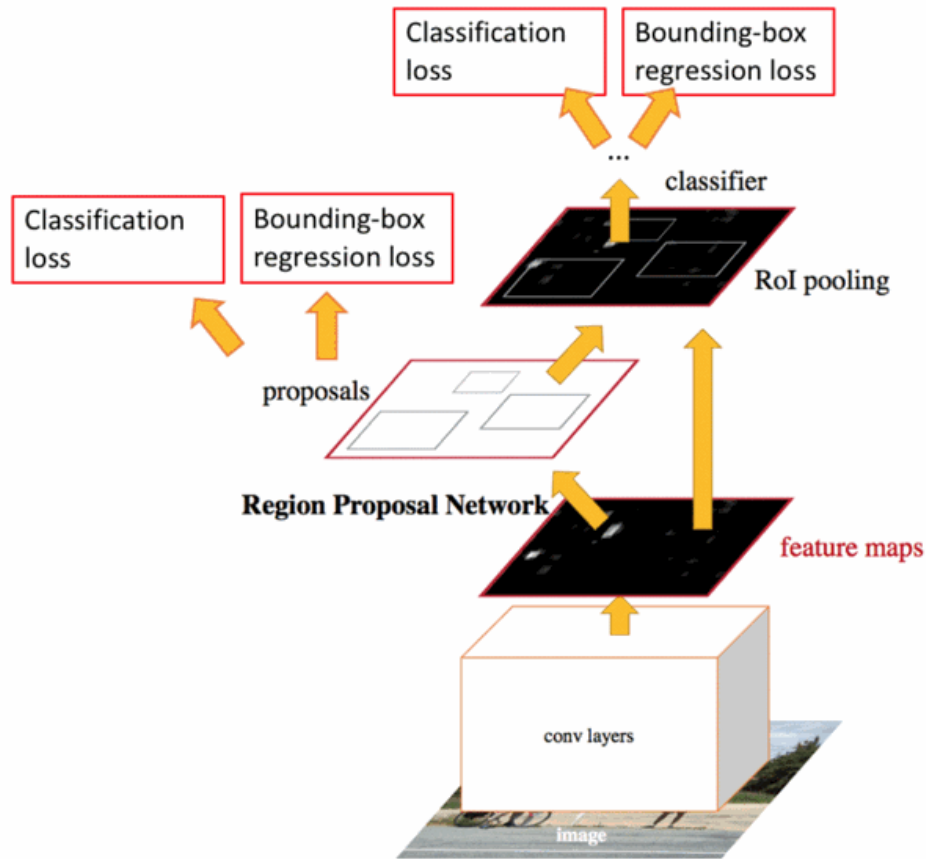
In YOLO, 845 boxes

In YOLO v3, 10,000 boxes



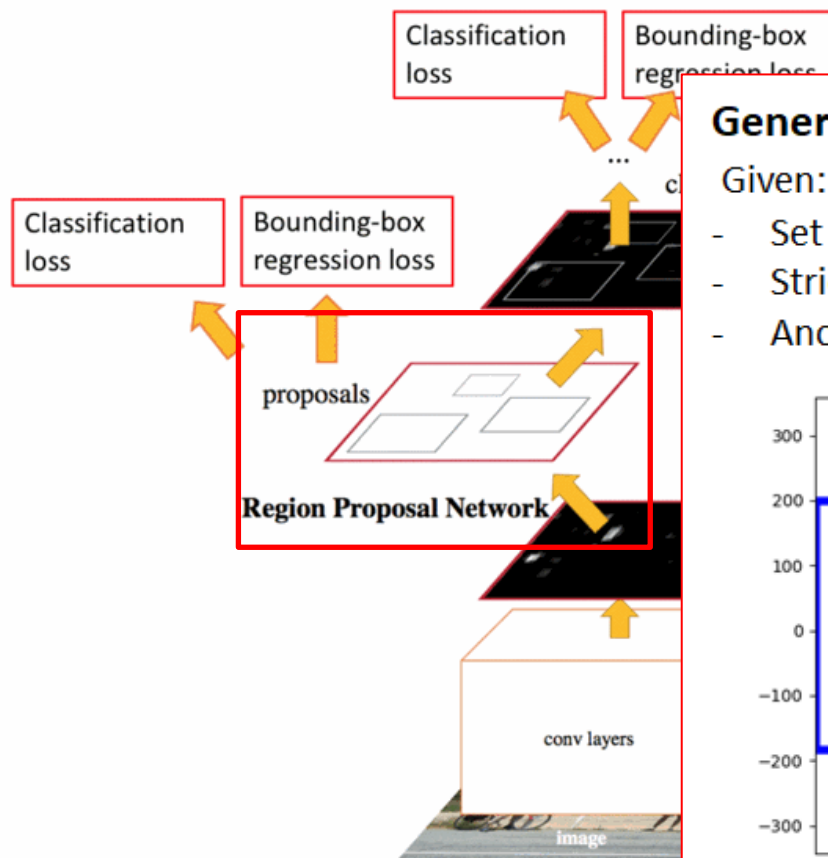
Dense-to-sparse method

❖ Faster R-CNN



Dense-to-sparse method

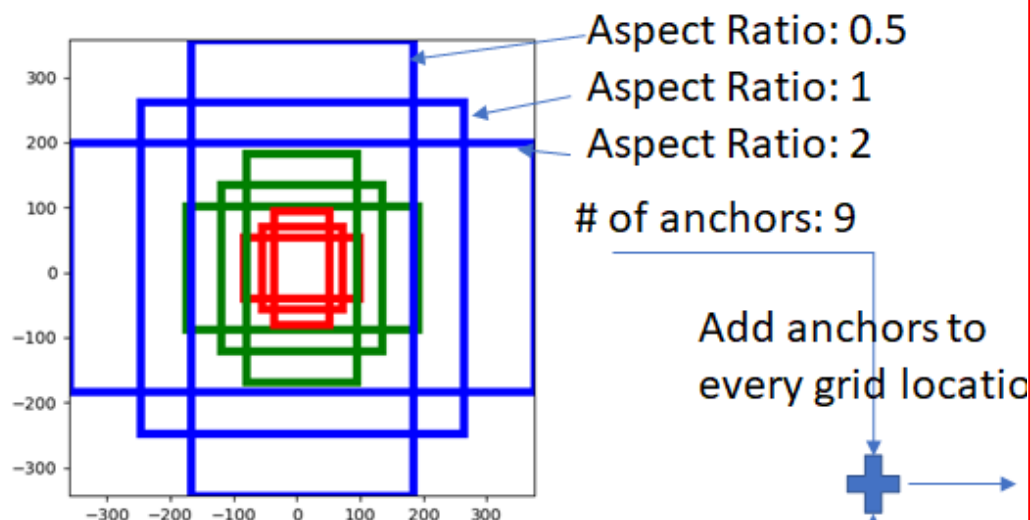
❖ Faster R-CNN



Generate Anchors

Given:

- Set of aspect ratios (0.5, 1, 2)
- Stride length (downscaling performed by resnet head: 16)
- Anchor Scales (8, 16, 32)

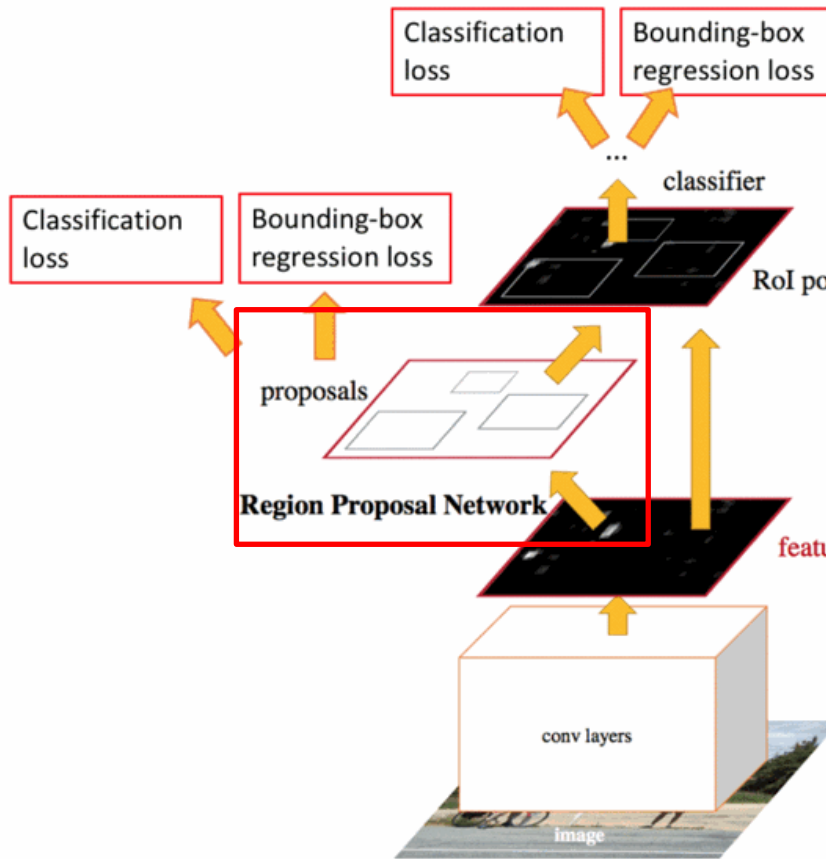


<https://techcommunity.microsoft.com/t5/educator-developer-blog/smart-acc-web-application-based-on-faster-rcnn-architecture/ba-p/381235>

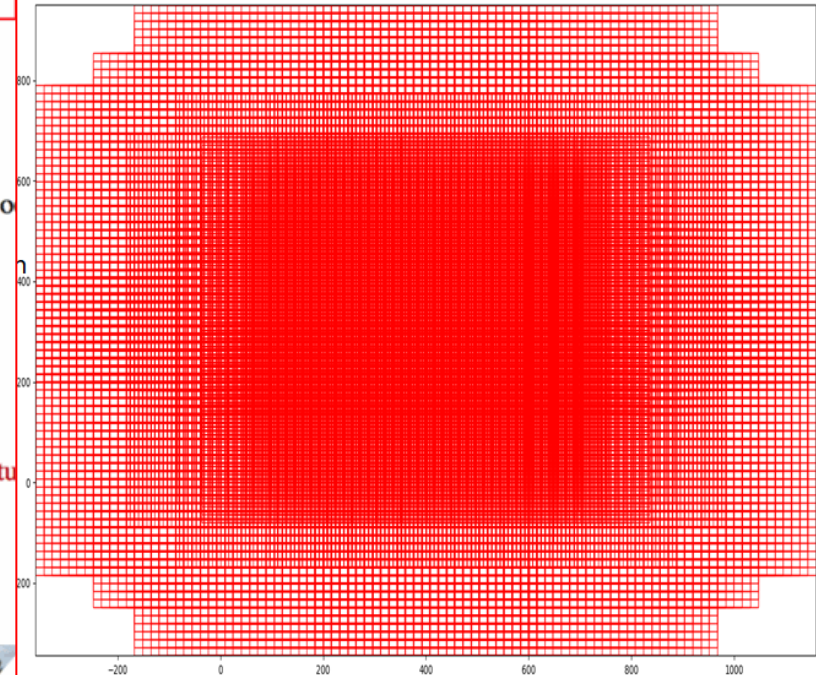
<https://www.telesens.co/2018/03/11/object-detection-and-classification-using-r-cnns/>

Dense-to-sparse method

❖ Faster R-CNN

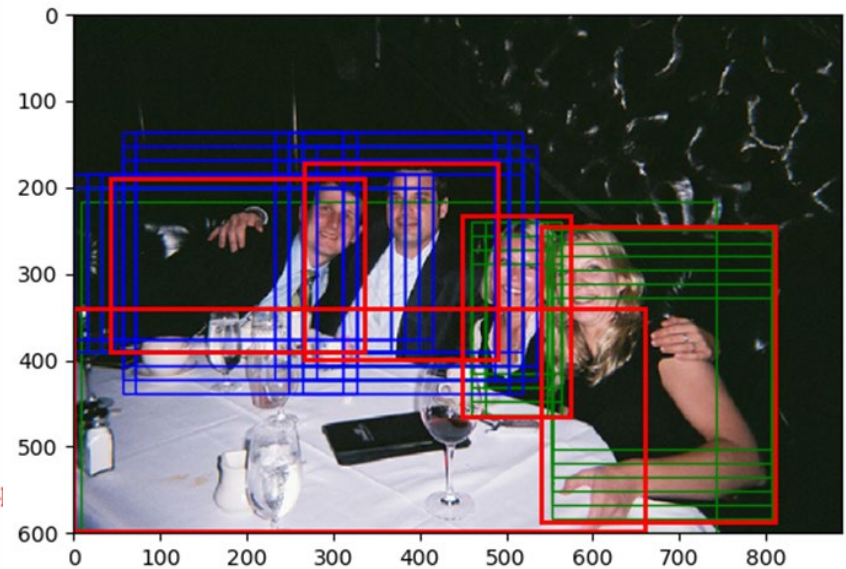
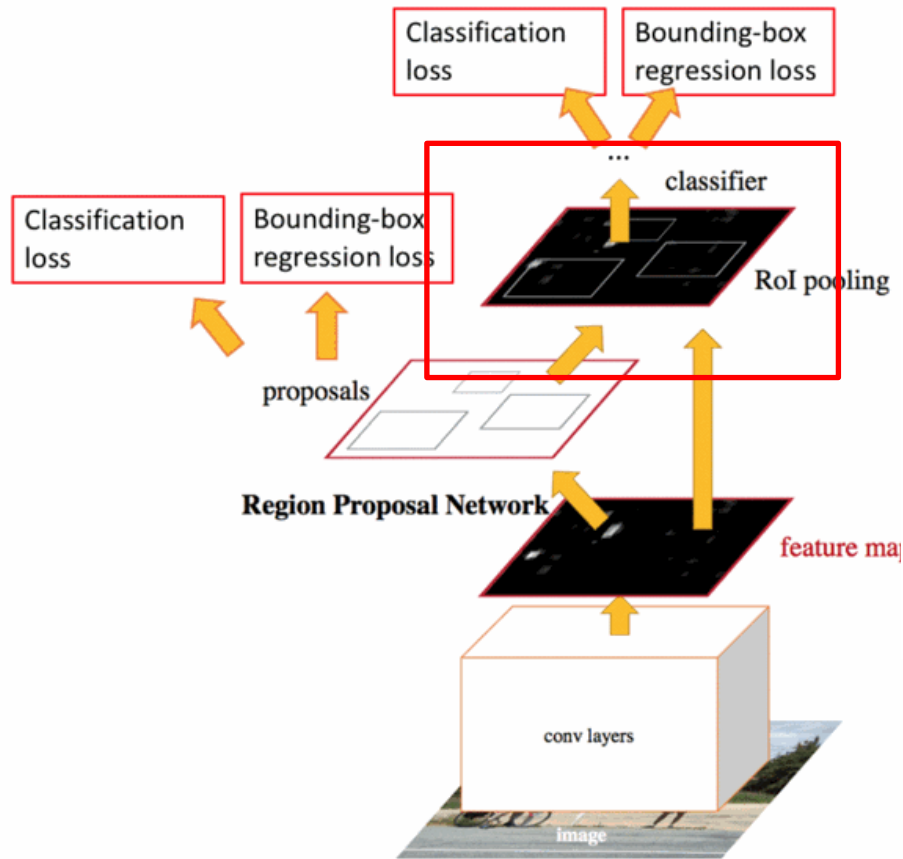


Total number of anchors: $1900 \times 9 = 17100$
Some boxes lie outside the image boundary



Dense-to-sparse method

❖ Faster R-CNN



Ground Truth boxes



Region Proposals

Limitations of Dense Region Proposals

1. Near-Duplicate Results

-> Post-processing techniques (Non-maximum suppression)

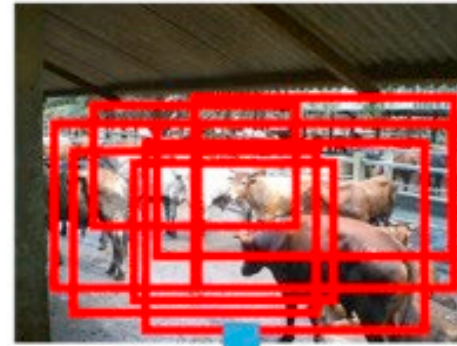
Before NMS



After NMS

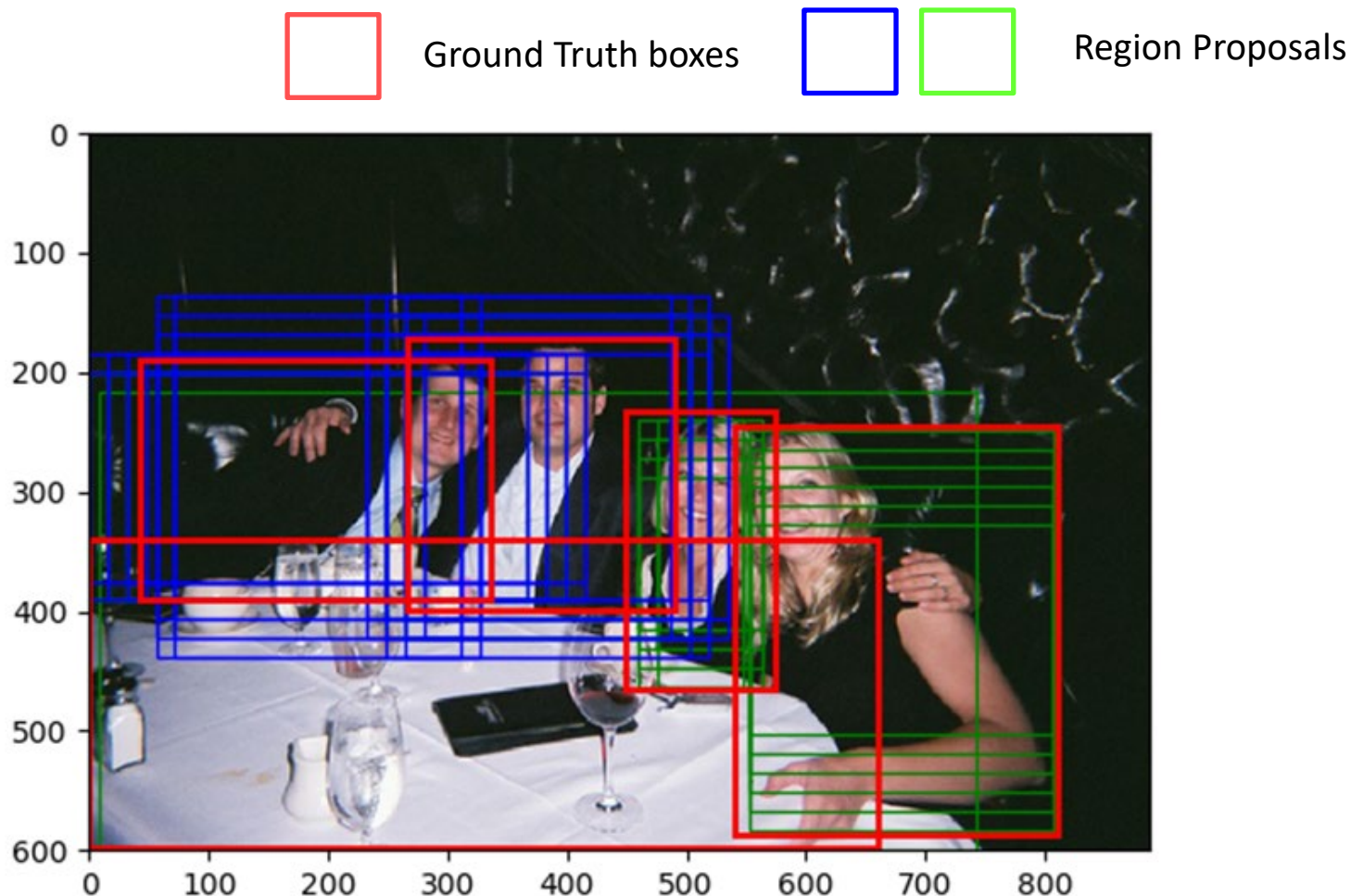


Overlapped objects are missing



Limitations of Dense Region Proposals

2. Many-to-one label assignment problem



Limitations of Dense Region Proposals

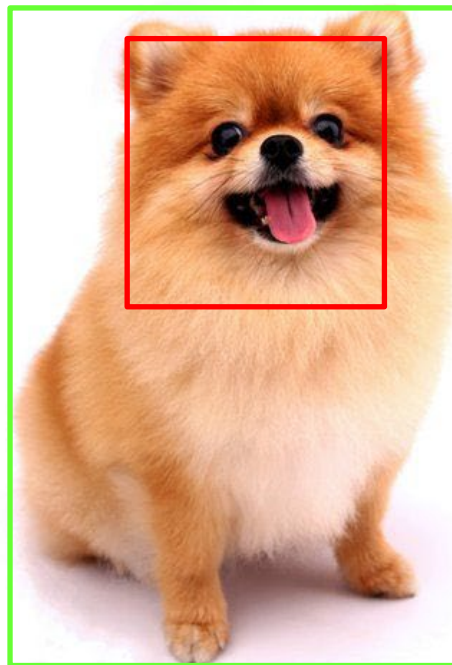
3. Sensitive to hyperparameters such as **box sizes**, aspect ratio, number of boxes, and density of reference point



Region Proposals



Ground Truth boxes



Limitations of Dense Region Proposals

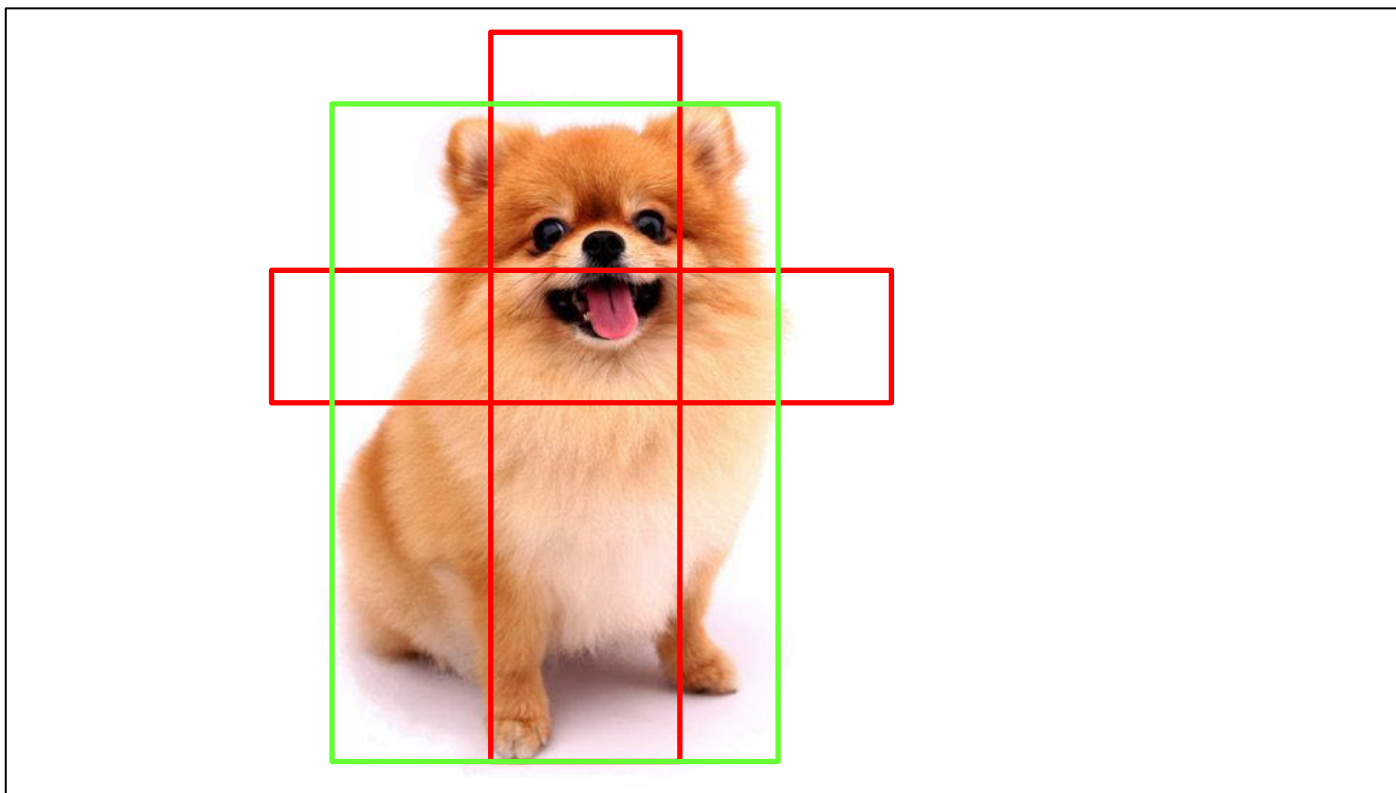
3. Sensitive to hyperparameters such as box sizes, **aspect ratio**, number of boxes, and density of reference point



Region Proposals



Ground Truth boxes



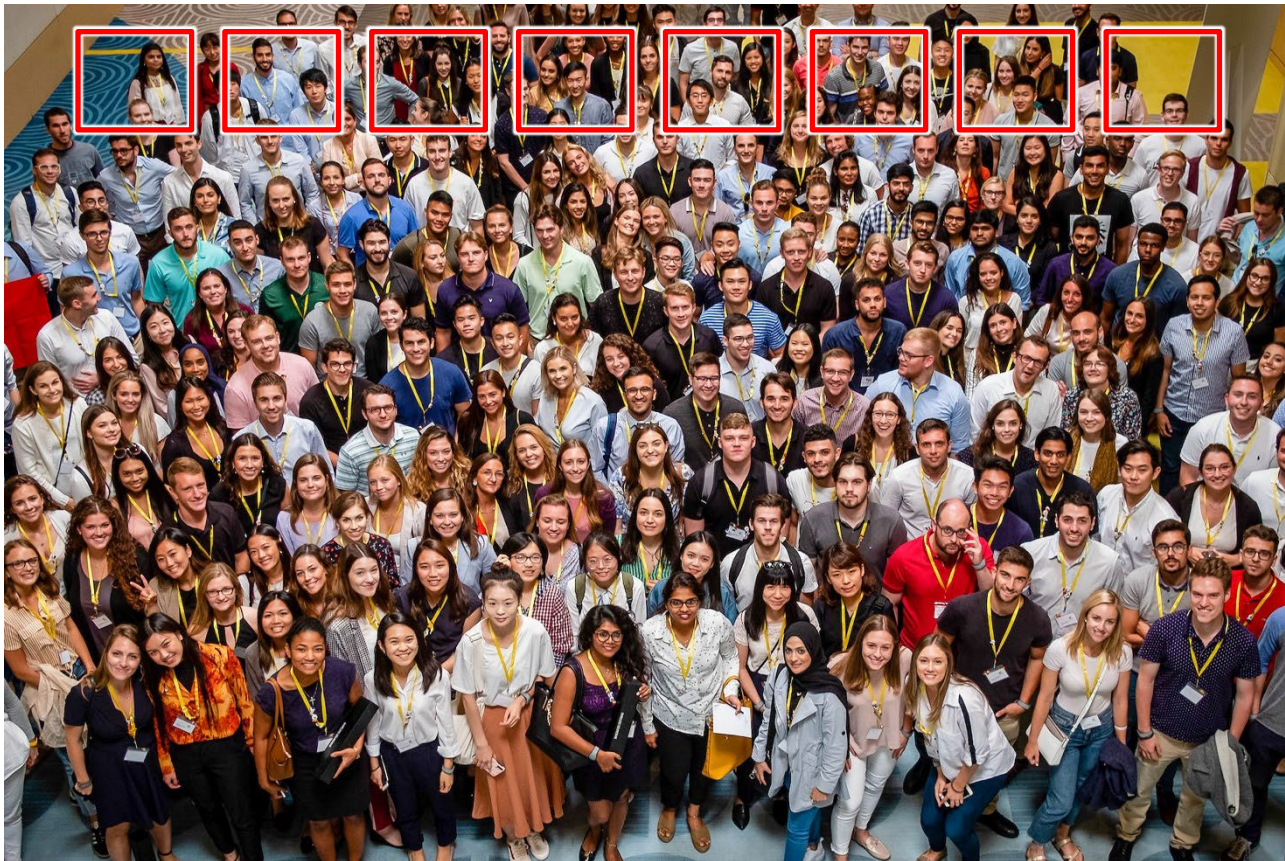
Limitations of Dense Region Proposals

3. Sensitive to hyperparameters such as box sizes, aspect ratio, **number of boxes**, and **density of reference point**



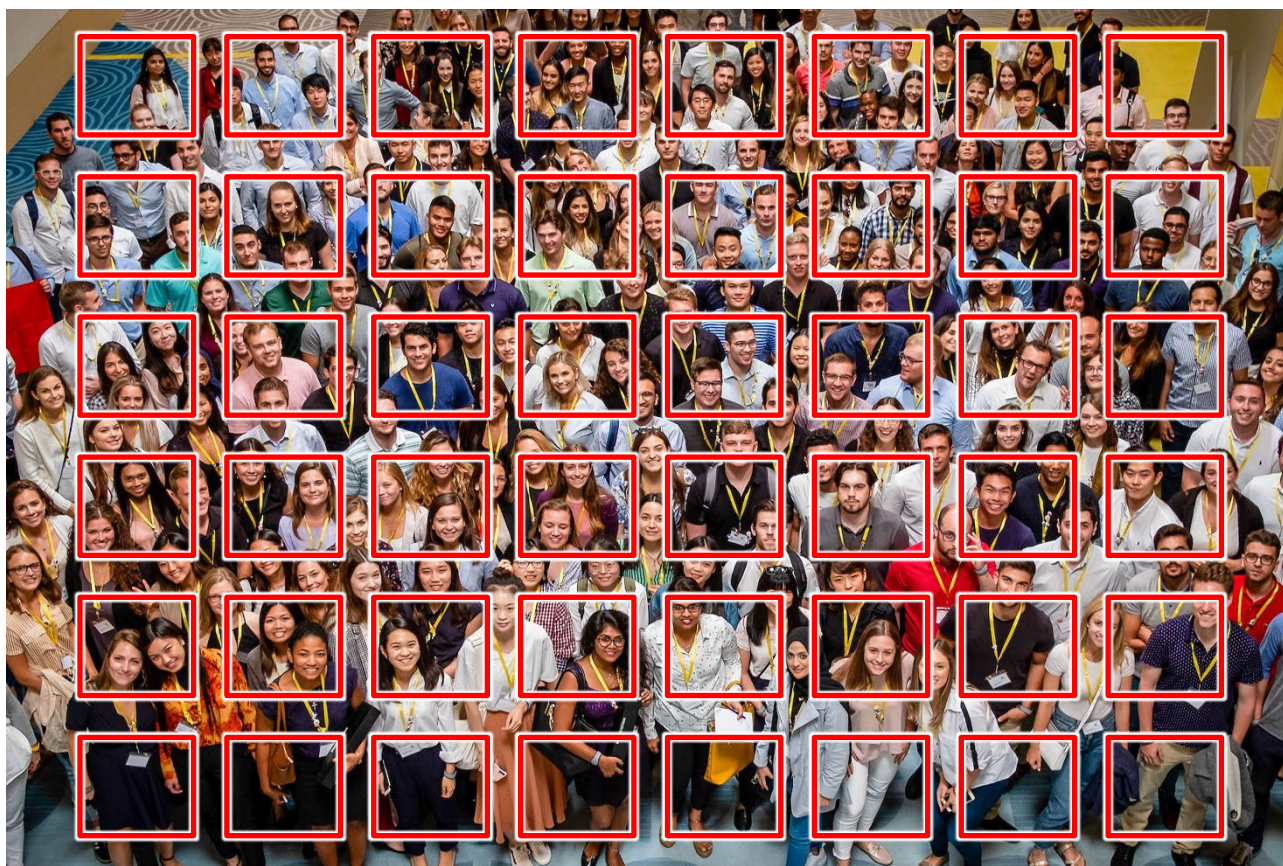
Limitations of Dense Region Proposals

3. Sensitive to hyperparameters such as box sizes, aspect ratio, **number of boxes**, and **density of reference point**



Limitations of Dense Region Proposals

3. Sensitive to hyperparameters such as box sizes, aspect ratio, **number of boxes**, and **density of reference point**





Problem to Solve!

Is it possible to design a
sparse detector?

Is it possible to design a **sparse** detector?

❖ Sparse boxes

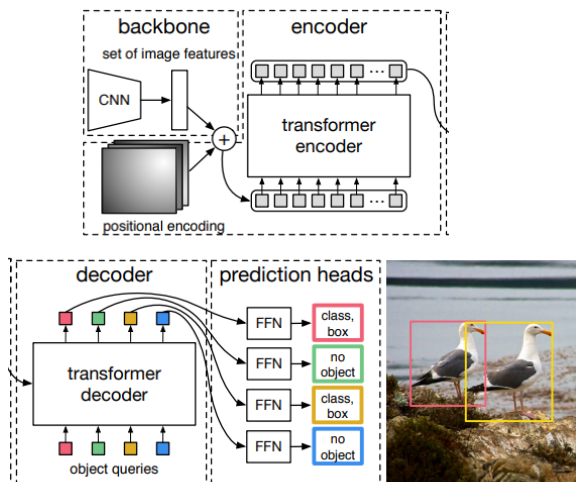
❑ Small number of starting boxes

- DETR

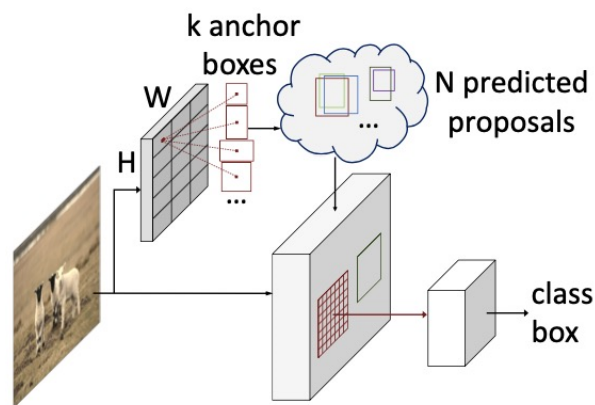
❖ Sparse features

❑ Local image feature for each bounding box

- Faster R-CNN



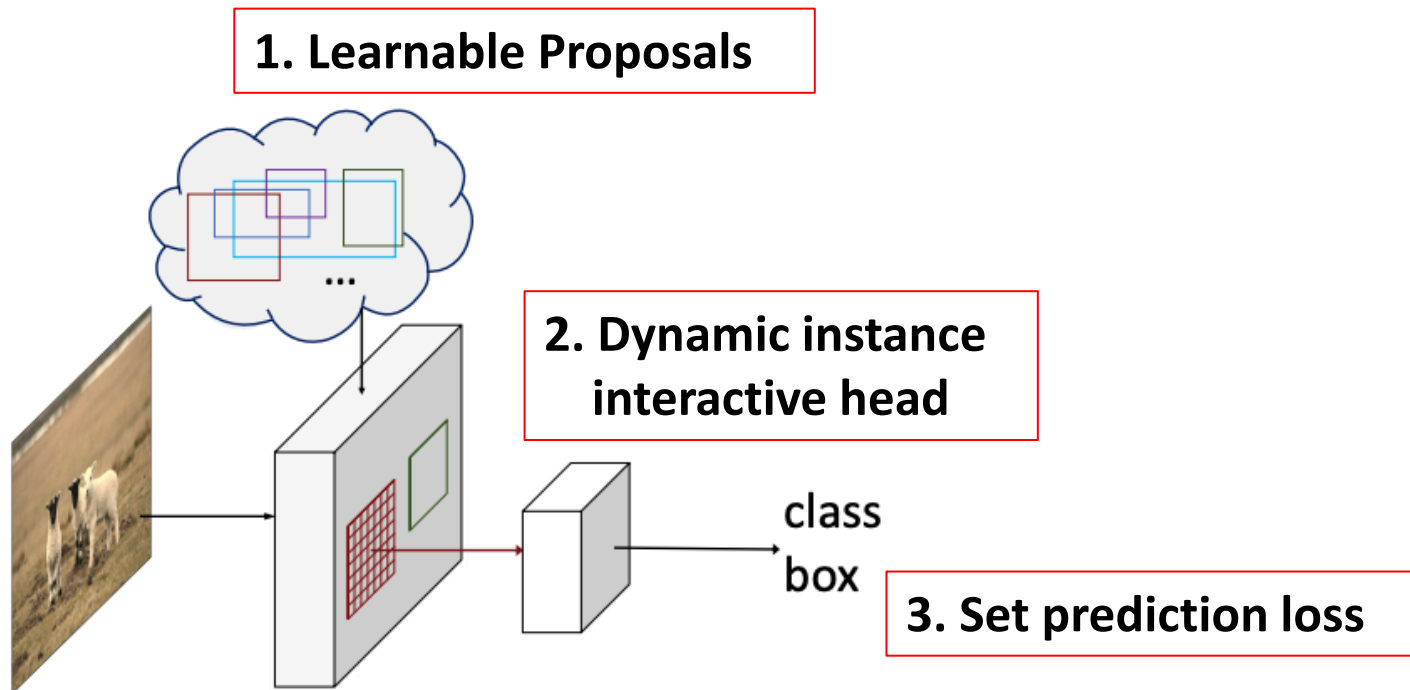
Sparse boxes : DETR



Sparse Features : Faster R-CNN

Sparse R-CNN

❖ Sparse-in and sparse-out object detector



(c) **Sparse:** Sparse R-CNN

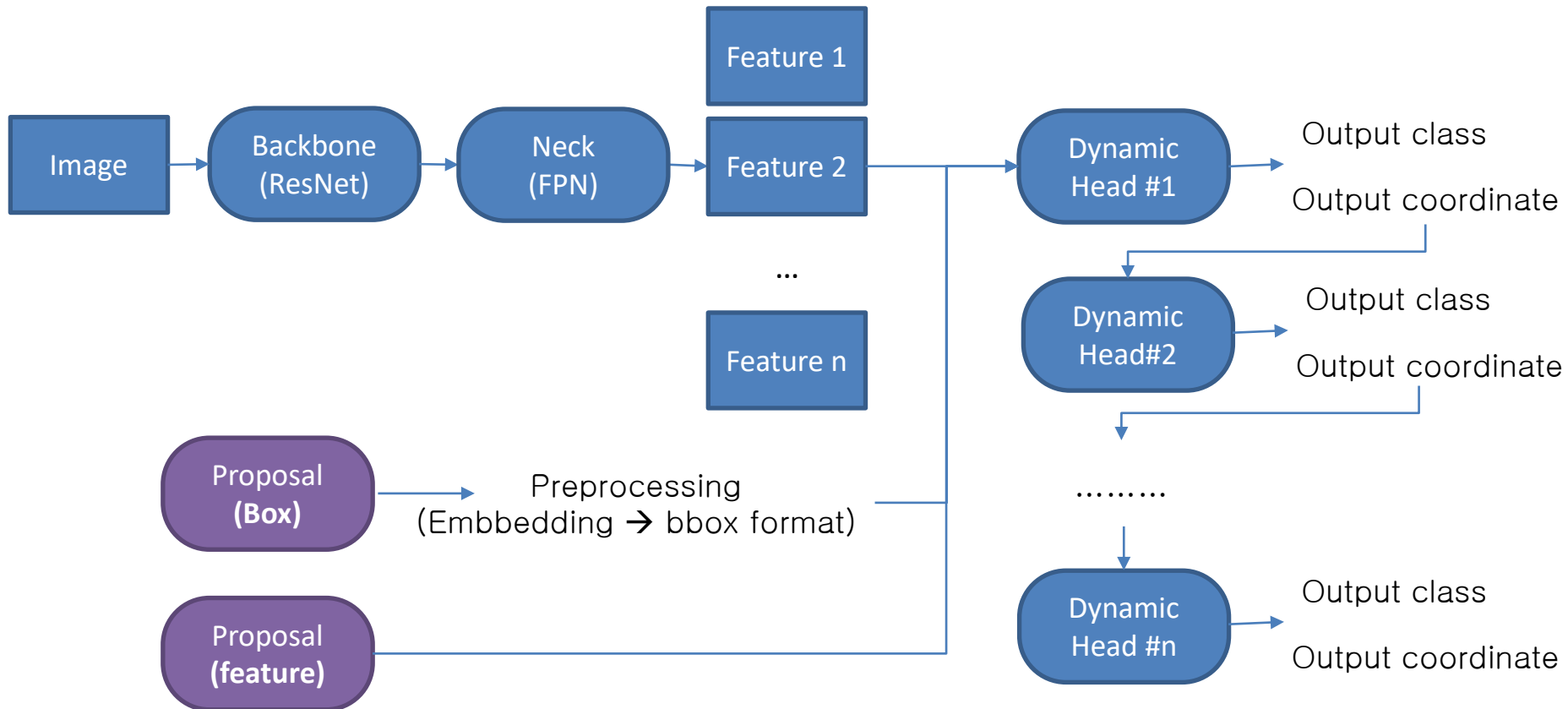


Q & A

Q & A

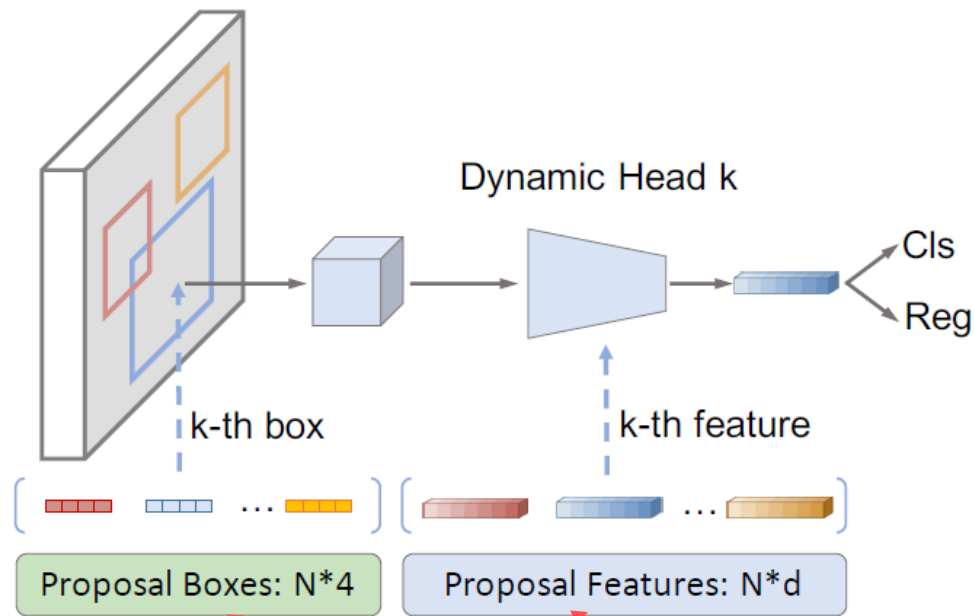


Overall layout of Sparse R-CNN



Sparse R-CNN

❖ Learnable proposal box & feature



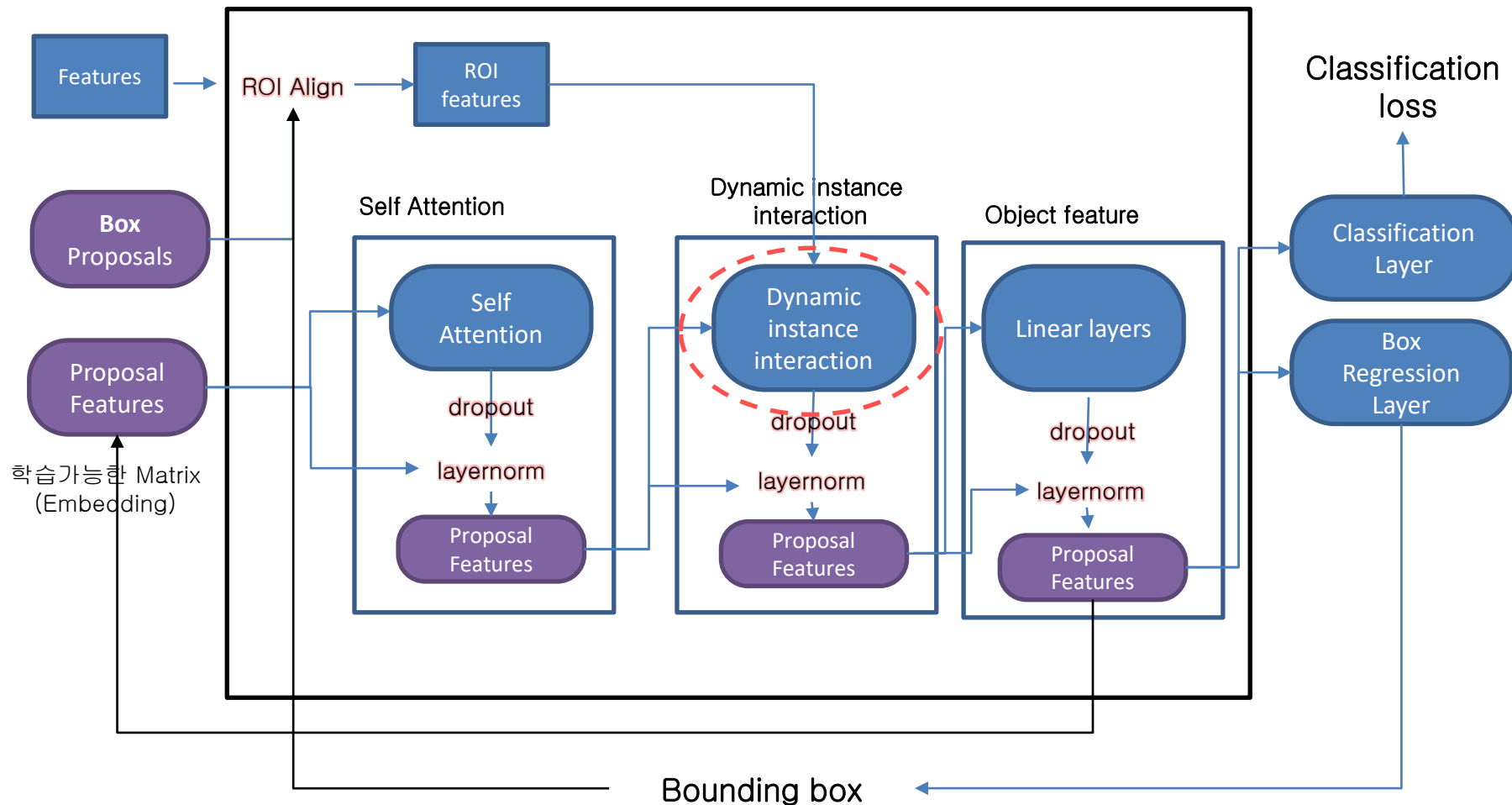
```
self.init_proposal_boxes = nn.Embedding(self.num_proposals, 4)
```

```
self.init_proposal_features = nn.Embedding(self.num_proposals, self.hidden_dim)
```

Sparse R-CNN

❖ Dynamic Head

Dynamic Head



❖ Dynamic Head

```
def dynamic_instance_interaction(pro_feats, roi_feats):
    # pro_feats: (N, C)
    # roi_feats: (N, S*S, C)

    # parameters of two 1x1 convs: (N, 2 * C * C/4)
    dynamic_params = linear1(pro_feats)
    # parameters of first conv: (N, C, C/4)
    param1 = dynamic_params[:, :C*C/4].view(N, C, C/4)
    # parameters of second conv: (N, C/4, C)
    param2 = dynamic_params[:, C*C/4:].view(N, C/4, C)

    # instance interaction for roi-features: (N, S*S, C)
    roi_feats = relu(norm(bmm(roi_feats, param1)))
    roi_feats = relu(norm(bmm(roi_feats, param2)))

    # roi_feats are flattened: (N, S*S*C)
    roi_feats = roi_feats.flatten(1)
    # obj_feats: (N, C)
    obj_feats = linear2(roi_feats)
    return obj_feats
```

Figure 4 – Pseudo-code of dynamic instance interaction, the k -th proposal feature generates dynamic parameters for the corresponding k -th RoI. bmm: batch matrix multiplication; linear: linear projection.



Sparse R-CNN

❖ Set prediction loss

$$\mathcal{L} = \lambda_{cls} \cdot \mathcal{L}_{cls} + \lambda_{L1} \cdot \mathcal{L}_{L1} + \lambda_{giou} \cdot \mathcal{L}_{giou}$$

Here $\mathcal{L}_{cls}$ is focal loss [23] of predicted classifications and ground truth category labels, $\mathcal{L}_{L1}$ and $\mathcal{L}_{giou}$ are L1 loss and generalized IoU loss [31] between normalized center coordinates and height and width of predicted boxes and ground truth box, respectively. λ_{cls} , λ_{L1} and λ_{giou} are coefficients of each component. The training loss is the same as the matching cost except that only performed on matched pairs. The final loss is the sum of all pairs normalized by the number of objects inside the training batch.



Q & A

Q & A



Experiments

❖ Experimental setup

Dataset. Our experiments are conducted on the challenging MS COCO benchmark [24] using the standard metrics for object detection. All models are trained on the COCO train2017 split (~118k images) and evaluated with val2017 (5k images).

Training details. ResNet-50 [14] is used as the backbone network unless otherwise specified. The optimizer is AdamW [26] with weight decay 0.0001. The mini-batch is 16 images and all models are trained with 8 GPUs. Default training schedule is 36 epochs and the initial learning rate is set to 2.5×10^{-5} , divided by 10 at epoch 27 and 33, respectively. The backbone is initialized with the pre-trained weights on ImageNet [7] and other newly added layers are initialized with Xavier [12]. Data augmentation includes random horizontal, scale jitter of resizing the input images such that the shortest side is at least 480 and at most 800

pixels while the longest at most 1333. Following [3, 49], $\lambda_{cls} = 2$, $\lambda_{L1} = 5$, $\lambda_{giou} = 2$. The default number of proposal boxes, proposal features and iteration is 100, 100 and 6, respectively. To stabilize training, the gradients are blocked at proposal boxes in each stage of iterative architecture, except initial proposal boxes.

Inference details. The inference process is quite simple in Sparse R-CNN. Given an input image, Sparse R-CNN directly predicts 100 bounding boxes associated with their scores. The scores indicate the probability of boxes containing an object. For evaluation, we directly use these 100 boxes without any post-processing.

Experiments

❖ Comparison with other detectors

Method	Feature	Epochs	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l	FPS
RetinaNet-R50 [42]	FPN	36	38.7	58.0	41.5	23.3	42.3	50.3	24
RetinaNet-R101 [42]	FPN	36	40.4	60.2	43.2	24.0	44.3	52.2	18
Faster R-CNN-R50 [42]	FPN	36	40.2	61.0	43.8	24.2	43.5	52.0	26
Faster R-CNN-R101 [42]	FPN	36	42.0	62.5	45.9	25.2	45.6	54.6	20
Cascade R-CNN-R50 [42]	FPN	36	44.3	62.2	48.0	26.6	47.7	57.7	19
DETR-R50 [3]	Encoder	500	42.0	62.4	44.2	20.5	45.8	61.1	28
DETR-R101 [3]	Encoder	500	43.5	63.8	46.4	21.9	48.0	61.8	20
DETR-DC5-R50 [3]	Encoder	500	43.3	63.1	45.9	22.5	47.3	61.1	12
DETR-DC5-R101 [3]	Encoder	500	44.9	64.7	47.7	23.7	49.5	62.3	10
Deformable DETR-R50 [49]	DeformEncoder	50	43.8	62.6	47.7	26.4	47.1	58.0	19
Sparse R-CNN-R50	FPN	36	42.8	61.2	45.7	26.7	44.6	57.6	23
Sparse R-CNN-R101	FPN	36	44.1	62.1	47.2	26.1	46.3	59.7	19
Sparse R-CNN*-R50	FPN	36	45.0	63.4	48.2	26.9	47.2	59.5	22
Sparse R-CNN*-R101	FPN	36	46.4	64.6	49.5	28.3	48.3	61.6	18

Table 1 – Comparisons with different object detectors on COCO 2017 val set. The top section shows results from Detectron2 [42] or original papers [3, 49]. Here “*” indicates that the model is with 300 learnable proposal boxes and random crop training augmentation, similar to Deformable DETR [49]. Run time is evaluated on NVIDIA Tesla V100 GPU.

Experiments

❖ Comparison with other detectors

Method	Backbone	TTA	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l
CornerNet [21]	Hourglass-104		40.6	56.4	43.2	19.1	42.8	54.3
CenterNet [48]	Hourglass-104		42.1	61.1	45.9	24.1	45.5	52.8
RepPoint [45]	ResNet-101-DCN		45.0	66.1	49.0	26.6	48.6	57.5
FCOS [36]	ResNeXt-101-DCN		46.6	65.9	50.8	28.6	49.1	58.6
ATSS [47]	ResNeXt-101-DCN	✓	50.7	68.9	56.3	33.2	52.9	62.4
YOLO [40]	CSPDarkNet-53		47.5	66.2	51.7	28.2	51.2	59.8
EfficientDet [34]	EfficientNet-B5		51.5	70.5	56.1	-	-	-
Sparse R-CNN	ResNeXt-101		46.9	66.3	51.2	28.6	49.2	58.7
Sparse R-CNN	ResNeXt-101-DCN		48.9	68.3	53.4	29.9	50.9	62.4
Sparse R-CNN	ResNeXt-101-DCN	✓	51.5	71.1	57.1	34.2	53.4	64.1

Table 2 – Comparisons with different object detectors on COCO 2017 test-dev set. The top section shows results from original papers. “TTA” indicates test-time augmentations, following the settings in [47].

So... is Sparse R-CNN SOTA??

❖ *No*

▣ COCO minival (papers with code)

Rank	Model	box AP ↑	AP50	AP75	APS	APM	APL	Extra Training Data	Paper	Code	Result	Year	Tags
1	SwinV2-G (HTC++)	62.5						×	Swin Transformer V2: Scaling Up Capacity and Resolution	GitHub	COCO	2021	
2	Florence-CoSwin-H	62						×	Florence: A New Foundation Model for Computer Vision		COCO	2021	
3	GLIP (Swin-L, multi-scale)	60.8						✓	Grounded Language-Image Pre-training	GitHub	COCO	2021	multiscale Vision Language Dynamic Head BERT-Base
4	Soft Teacher+Swin-L (HTC++, multi-scale)	60.7						✓	End-to-End Semi-Supervised Object Detection with Soft Teacher	GitHub	COCO	2021	multiscale Swin-Transformer
35	Sparse R-CNN (PVTv2-B2)	50.1	69.5	54.9				×	PVTv2: Improved Baselines with Pyramid Vision Transformer	GitHub	COCO	2021	



Experiments

❖ Ablation studies

Sparse	Iterative	Dynamic	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l
✓			18.5	35.0	17.7	8.3	21.7	26.4
✓	✓		32.2 (+13.7)	47.5 (+12.5)	34.4 (+16.7)	18.2 (+9.9)	35.2 (+13.5)	41.7 (+15.3)
✓	✓	✓	42.3 (+10.1)	61.2 (+13.7)	45.7 (+11.3)	26.7 (+8.5)	44.6 (+9.4)	57.6 (+15.9)

Table 3 – Ablation studies on each components in Sparse R-CNN. Starting from Faster R-CNN, we gradually add learnable proposal boxes, iterative architecture, and dynamic head in Sparse R-CNN. All models are trained with set prediction loss.



Experiments

❖ Ablation studies

Cascade	Feature reuse	AP	AP ₅₀	AP ₇₅
		18.5	35.0	17.7
✓		20.5(+2.0)	29.3	20.7
✓	✓	32.2(+11.7)	47.5	34.4

Table 4 – The effect of feature reuse in iterative architecture. Original cascading implementation makes no big difference. Concatenating object feature of previous stage to object feature of current stage leads to a huge improvement.

Self-att.	Ins. interact	AP	AP ₅₀	AP ₇₅
		32.2	47.5	34.4
✓		37.2(+5.0)	54.8	40.1
✓	✓	42.3(+5.1)	61.2	45.7

Table 5 – The effect of instance-interaction in dynamic head. Without instance interaction, dynamic head degenerates to self-attention. The gain comes from both self-attention and instance-interaction.



Experiments

Init.	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l
Center	41.5	59.6	45.0	25.6	43.9	56.1
Image	42.3	61.2	45.7	26.7	44.6	57.6
Grid	41.0	59.4	44.2	23.8	43.7	55.6
Random	42.1	60.3	45.3	24.5	44.6	57.9

Table 6 – Effect of initialization of proposal boxes. Detection performance is relatively robust to initialization of proposal boxes.

- “Center” means all proposal boxes are located in the center of image at beginning, height and width is set to 0.1 of image size.
- “Image” means all proposal boxes are initialized as the whole image size.
- “Grid” means proposal boxes are initialized as regular grid in image, which is exactly the initial boxes in G-CNN [27].
- “Random” denotes the center, height and width of proposal boxes are randomly initialized with Gaussian



Experiments

Proposals	AP	AP ₅₀	AP ₇₅	FPS	Training time
100	42.3	61.2	45.7	23	19h
300	43.9	62.3	47.4	22	24h
500	44.6	63.2	48.5	20	60h

Table 7 – Effect of number of proposals. Increasing number of proposals leads to continuous improvement, while more proposals take more training time.



Experiments

Stages	AP	AP ₅₀	AP ₇₅	FPS	Training time
1	21.7	36.7	22.3	35	12h
2	36.2	52.8	38.8	33	13h
3	39.9	56.8	43.2	29	15h
6	42.3	61.2	45.7	23	19h
12	41.6	60.2	45.0	17	30h

Table 8 – Effect of number of stages. Gradually increasing the number of stages, the performance is saturated at 6 stages.



Experiments

Method	AP	AP ₅₀	AP ₇₅
Multi-head Attention [38]	35.7	54.9	37.7
Dynamic head	42.3(+6.6)	61.2	45.7

Table 9 – Dynamic head vs. Multi-head Attention. As object recognition head, dynamic head outperforms multi-head attention.



Experiments

Method	Pos. encoding	AP	AP ₅₀	AP ₇₅
DETR [3]	✓	40.6	61.6	-
DETR [3]		32.8 (-7.8)	55.2	-
Sparse R-CNN	✓	41.9	60.9	45.0
Sparse R-CNN		42.3(+0.4)	61.2	45.7

Table 10 – Proposal feature vs. Object query. Object query is learned positional encoding, while proposal feature is irrelevant to position.



Q & A

Q & A



경청해주셔서 감사합니다!

Thanks for your attention!!